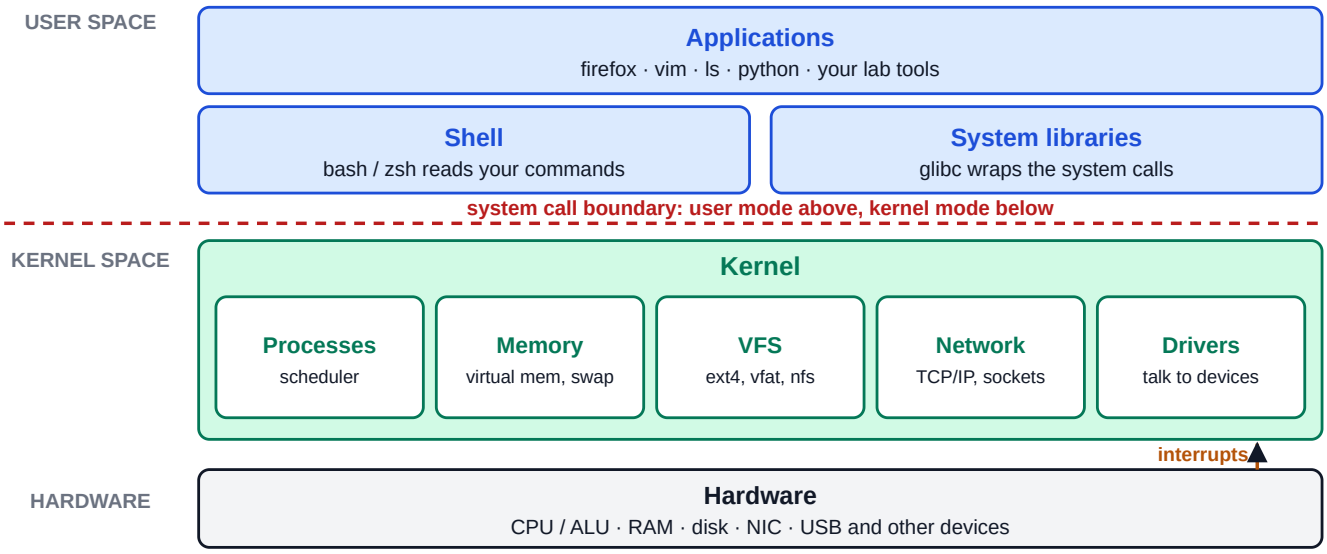
OPERATING SYSTEMS & SECURITY · MODULE 1

Linux Lab Cheat Sheet

Abstraction, virtualization, OS types and Linux file systems, turned into commands and diagrams to keep open during labs. Commands that change things are marked with `sudo`. Run them on a VM, and take a snapshot first.

1. The big picture: layers of an OS



The OS is an abstraction: each layer hides the messier one below it. Programs never touch hardware directly. They make a **system call**, the CPU flips to **kernel mode**, the kernel does the work, then flips back to **user mode**.

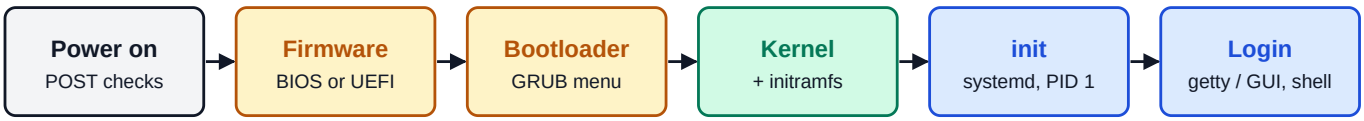
WHO AM I RUNNING ON?

`uname -a`	kernel version, hostname, CPU arch
`uname -m / arch`	CPU architecture: `x86_64` or `aarch64` (ARM)
`lscpu`	CPU details (cores, arch, virtualization flags)
`cat /etc/os-release`	which distro and version
`echo $SHELL ; ps -p $$`	which shell you're in

KERNEL, MODULES, SYSCALLS

`lsmod`	kernel modules (drivers) loaded right now
`modinfo <module>`	details about one module
`sudo modprobe <module>`	load a module (`-r` removes it)
`strace -c ls`	count the system calls a command makes
`cat /proc/interrupts`	interrupt counts per CPU and device

2. Boot process




```
Check which firmware booted the system: [ -d /sys/firmware/efi ] && echo UEFI || echo BIOS
```

Firmware wakes the hardware, GRUB picks and loads the kernel, the kernel starts **systemd** (PID 1), and systemd starts everything else.

INSPECT THE BOOT

`dmesg | less`	kernel messages from this boot (hardware, drivers)
`journalctl -b`	full log for this boot (`-b -1` = last boot)
`journalctl -k`	kernel messages only
`systemctl get-default`	boot target: graphical or multi-user (CLI)
`systemd-analyze blame`	what slowed the boot down

GRUB

`/etc/default/grub`	GRUB settings (timeout, kernel options)
`sudo update-grub`	apply changes (Debian/Ubuntu)
`sudo grub2-mkconfig -o /boot/grub2/grub.cfg`	apply changes (RHEL/Fedora)
`ls /boot`	kernels (vmlinuz-*) and initramfs images

	BIOS + MBR (legacy)	UEFI + GPT (modern)
Partitions	4 primary (or 3 primary + 1 extended with logicals)	128 by default
Max disk size	2 TB	about 9.4 ZB (no practical limit)
Tool	`fdisk`	`gdisk`, `parted` (and modern `fdisk`)
Security	boot sector is a classic malware target (bootkits)	Secure Boot checks signatures before loading
Boot code lives in	first 512-byte sector of the disk	EFI system partition (vfat), mounted at `/boot/efi`

3. The Linux directory tree

```
/                    root of everything (one tree, no drive letters)
├── bin  -> usr/bin  everyday commands (ls, cp, cat)
├── boot             kernel (vmlinuz), initramfs, GRUB files
├── dev              device files (sda, sda1, tty, null, random)
├── etc              system config (fstab, passwd, shadow, group)
├── home             user folders (/home/tayo)
├── lib  -> usr/lib  shared libraries and kernel modules
├── media            auto-mounted USBs and DVDs
├── mnt              where you mount things by hand in labs
├── opt              optional third-party software
├── proc             live kernel/process info (not on disk)
├── root             the root user's home folder
├── run              runtime data since this boot (PID files)
├── sbin -> usr/sbin admin commands (fdisk, mkfs, fsck)
├── srv              data a server hands out (web, ftp)
├── sys              live hardware/driver info (not on disk)
├── tmp              temp files, everyone can write (sticky bit)
├── usr              installed programs, libraries, docs
└── var              changing data
    ├── log          logs: syslog/messages, auth.log or secure
    └── tmp          temp files kept across reboots
```

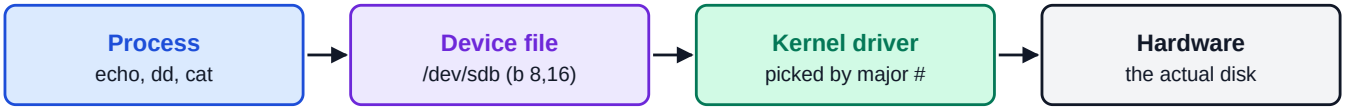
Everything is a file. Disks, terminals and even running processes show up as files: `/dev/sda`, `/dev/tty1`, `/proc/1234/`.

Lab-useful files

`/etc/passwd` users
`/etc/shadow` password hashes (root only)
`/etc/group` groups
`/etc/fstab` what mounts at boot
`/etc/sudoers` who can sudo (edit with `visudo`)
`/var/log/auth.log` logins (Debian/Ubuntu)
`/var/log/secure` logins (RHEL/Fedora)
`/proc/cpuinfo`, `/proc/meminfo`

`tree -L 1 /`	draw this tree yourself (install `tree`)
`ls -la /`	list it with permissions

4. Disks, partitions and device files



ls -l /dev: first letter b = block device (disks, data in blocks) c = character device (tty, data as a byte stream)

A **device file** is the interface between a process and a driver. Write to `/dev/sdb` and the kernel driver writes to the disk. New device files appear automatically (udev) when hardware connects.

GPT DISK, AS `LSBLK` DRAWS IT

```
$ lsblk
NAME     SIZE TYPE MOUNTPOINTS
sda       64G disk                <- whole disk (GPT, UEFI)
├─sda1   512M part /boot/efi      <- EFI system partition (vfat)
├─sda2    59G part /              <- root filesystem (ext4)
└─sda3   4.5G part [SWAP]         <- swap partition
sdb       20G disk                <- second disk you add in a lab
└─sdb1    20G part /mnt/data      <- you partition, format, mount it
sr0     1024M rom                 <- CD/DVD (ISO)
```

MBR DISK WITH EXTENDED + LOGICAL

```
MBR disk (/dev/sdc): 4 primary slots max
├─sdc1  primary
├─sdc2  primary
├─sdc3  primary
└─sdc4  extended  (a container, holds no data itself)
    ├─sdc5  logical   <- logical partitions always start at 5
    └─sdc6  logical
```

LOOK BEFORE YOU TOUCH

`lsblk`	tree of disks and partitions
`lsblk -f`	same, plus filesystem type, UUID, mountpoint
`sudo blkid`	UUIDs and filesystem types
`sudo fdisk -l`	every disk and its partition table
`ls -l /dev/sd*`	device files (b = block)
`sudo mknod /dev/x c 1 3`	make a device file by hand (type, major, minor)

NAMING

`sda` = 1st disk, `sdb` = 2nd. `sda1` = 1st partition. NVMe: `nvme0n1p1`. VirtualBox/KVM virtio disks: `vda`.

5. Partition, format, mount: the lab workflow

1. **Find the disk** `lsblk`. Make sure it is the new empty one (for example `/dev/sdb`), not your OS disk.
2. **Partition** `sudo fdisk /dev/sdb` then `n` new, `p` primary, Enter for defaults, `p` print, `w` write. (`m` = help, `q` = quit without saving.) For GPT use `sudo gdisk /dev/sdb` (same keys).
3. **Format** `sudo mkfs -t ext4 /dev/sdb1` (same as `mkfs.ext4`)
4. **Mount** `sudo mkdir -p /mnt/data` then `sudo mount /dev/sdb1 /mnt/data`
5. **Check** `df -h /mnt/data` or `findmnt /mnt/data`
6. **Make it permanent** add a line to `/etc/fstab`, then test with `sudo mount -a` (an fstab typo can stop the next boot)

HOW A MOUNT ATTACHES TO THE TREE

```
/                       (sda2, ext4)
├── home
├── boot
│   └── efi             <- sda1 (vfat) mounted here
└── mnt
    └── data            <- sdb1 (ext4) mounted here
        └── lab1.txt       (really stored on sdb1)
```

/ETC/FSTAB LINE, FIELD BY FIELD

```
UUID=1a2b...  /mnt/data  ext4  defaults  0  2
```

Field	Example	Meaning
1	`UUID=1a2b...`	which device
2	`/mnt/data`	mount point

3	`ext4`	filesystem type
4	`defaults`	mount options
5	`0`	dump backup (0 = off)
6	`2`	fsck order at boot

fsck order: `1` = root filesystem, `2` = other disks, `0` = never check. Use the UUID from `blkid`: `sdb` can change names between boots.

`sudo umount /mnt/data`	unmount (leave the folder first, or you get "target is busy")
`parted /dev/sdb print`	partition table, MBR or GPT
`sudo gparted`	GUI partition editor

6. Filesystems, inodes, checks and swap

Type	Use it for
`ext4`	default Linux filesystem, journaling (ext3 added the journal, ext2 has none)
`xfs`	RHEL/Fedora default for large disks
`vfat`	FAT32: USB sticks, EFI partition, Windows-compatible
`ntfs`	Windows disks
`nfs`	Network File System: share folders between Linux hosts
`smb/cifs`	Samba: share with Windows
`swap`	overflow for RAM (not a real filesystem)

INODES

Every file has an **inode**: a record of its owner, permissions, size, timestamps, link count and where its data blocks are. The **filename is not in the inode**. It lives in the directory entry that points to it. The **superblock** describes the whole filesystem (size, block count, inode count).

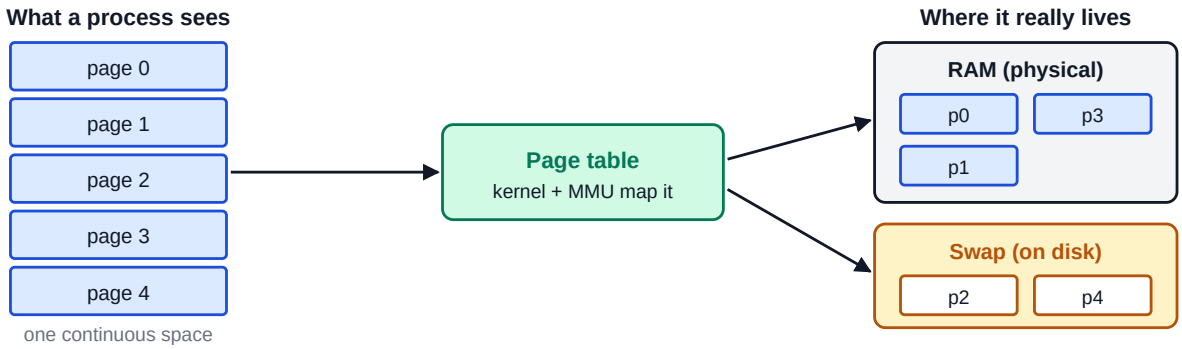
`ls -i file`	inode number
`stat file`	everything in the inode
`df -i`	inodes used vs free per filesystem
`df -h`	disk space used vs free

CHECK AND REPAIR

`sudo umount /dev/sdb1`	ALWAYS unmount before a manual check
`sudo fsck /dev/sdb1`	check and repair (picks the right checker)
`sudo e2fsck -f /dev/sdb1`	force a check on ext2/3/4
`sudo dosfsck /dev/sdc1`	FAT/vfat (also `fsck.vfat`)
`sudo tune2fs -l /dev/sdb1`	show ext settings (mount count, last check)
`sudo tune2fs -c 30 /dev/sdb1`	force a check every 30 mounts

MEMORY AND SWAP

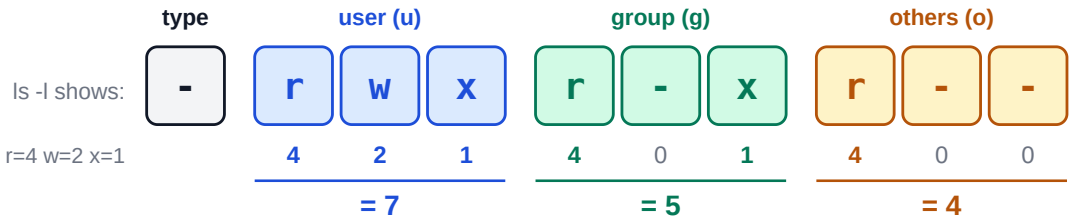
`free -h`	RAM and swap in use
`swapon --show`	active swap areas
`sudo mkswap /dev/sdb2`	turn a partition into swap
`sudo swapon /dev/sdb2`	start using it (`swapoff` stops it)
`vmstat 1 5`	memory, swap in/out (si/so), CPU every 1s



Inactive pages get pushed to swap when RAM runs low. Check: free -h swapon --show

Virtual memory: each process thinks it has one continuous block. The kernel maps pages to RAM or, when RAM is short, to swap.

7. Permissions



-rwxr-xr-- = chmod 754 = chmod u=rwx,g=rx,o=r

Read each trio as a sum. r = 4, w = 2, x = 1, and a dash is 0.

Number	Shows as	Typical use
`777`	`rwxrwxrwx`	never (anyone can change it)
`755`	`rwxr-xr-x`	scripts, programs, folders
`750`	`rwxr-x---`	team folder, no outsiders
`700`	`rwx------`	private folder, `~/.ssh`
`666`	`rw-rw-rw-`	everyone reads and writes (= `a+rw`)
`644`	`rw-r--r--`	normal files (= `u=rw,go=r`)
`600`	`rw-------`	private keys, secrets
`640`	`rw-r-----`	`/etc/shadow`-style files

On folders, r = list names, w = create/delete files inside, x = enter it (`cd`). A script needs r *and* x. A compiled binary can run with x only.

CHANGE THEM

`chmod 755 run.sh`	set with numbers
`chmod u+x run.sh`	add execute for the user (owner)
`chmod go-w file`	remove write from group and others
`chmod a+r file`	everyone can read (a = all)
`chmod -R 750 dir/`	apply to a folder and everything in it
`sudo chown tayo:staff file`	change owner and group
`umask`	default mask: 022 means new files 644, folders 755
`id / groups`	your user ID and groups
`sudo -l`	what you're allowed to run with sudo

u = user (owner) · **g** = group · **o** = others · **a** = all.
o is others, not owner.

8. Special bits: SUID, SGID, sticky

Bit	Number	Set with	Shows as	What it does
SUID	`4xxx`	`chmod u+s`	`rwsr-xr-x`	runs as the file's **owner** (often root), not as you
SGID	`2xxx`	`chmod g+s`	`rwxr-sr-x`	file: runs as the group. folder: new files inherit the folder's group
Sticky	`1xxx`	`chmod +t`	`rwxrwxrwt`	folder: only the file's owner, the folder's owner or root can delete files

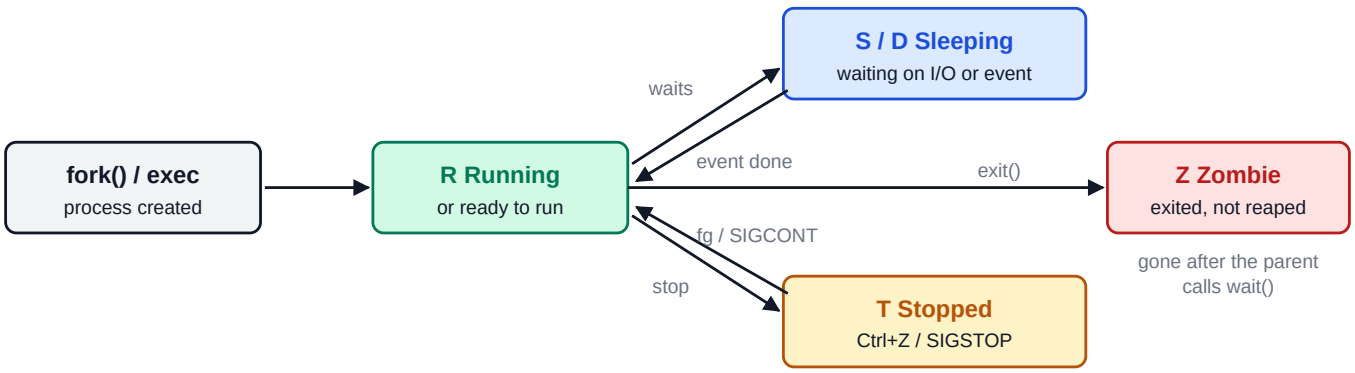
Capital `S` or `T` means the bit is set but execute is *not*, which is usually a mistake. Real examples: `/usr/bin/passwd` is SUID root (that's how you can change your own hash in `/etc/shadow`). `/tmp` is `1777`.

HUNT FOR THEM (PRIVILEGE ESCALATION CHECKS)

`find / -perm -4000 -type f 2>/dev/null`	all SUID files
`find / -perm -2000 -type f 2>/dev/null`	all SGID files
`find / -perm /6000 -type f 2>/dev/null`	SUID **or** SGID
`find / -perm -6000 -type f 2>/dev/null`	SUID **and** SGID together (rare)
`find / -type d -perm -0002 ! -perm -1000 2>/dev/null`	world-writable folders **without** sticky bit (risky)
`ls -ld /tmp`	should end in `t`

Why it matters: an unexpected SUID-root binary is one of the first things attackers (and you, in a lab) look for. If it can run a shell or edit files, it can give root. Compare what you find against GTFOBins.

9. Processes



`ps -eo pid,stat,cmd` (the STAT column shows these letters)

Linux process states. The scheduler switches the CPU between ready processes many times a second (context switching).

SEE THEM

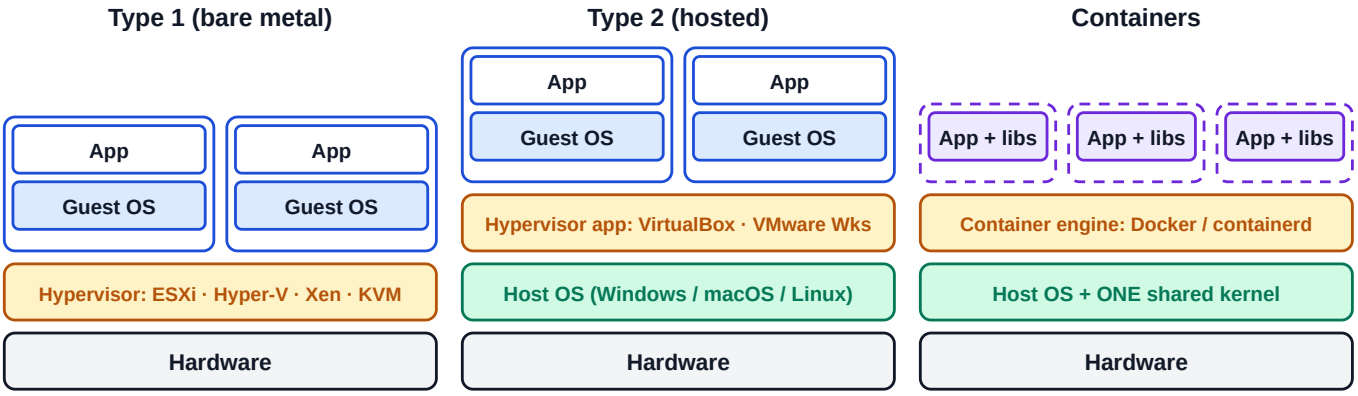
`ps aux`	every process: user, PID, %CPU, %MEM, command
`ps -ef --forest`	processes as a parent/child tree
`pstree -p`	tree view starting from systemd (PID 1)
`top / htop`	live view (`q` quits, `k` kills in top)
`pgrep -a sshd`	find processes by name

CONTROL THEM

`kill <PID>`	ask nicely (SIGTERM, 15)
`kill -9 <PID>`	force it (SIGKILL), no cleanup
`pkill <name>`	kill by name
`cmd & / jobs / fg / bg`	run in background and manage jobs
`nice -n 10 cmd / renice 5 -p <PID>`	lower priority (higher number = nicer)

`ls /proc/<PID>/`	everything the kernel knows about a process

10. Virtualization and containers



Type 1 runs right on the hardware (servers, cloud). Type 2 runs as an app on your OS (your home lab). Containers share the host's kernel, so they're lighter but less isolated than VMs.

WHERE AM I RUNNING?

`systemd-detect-virt`	am I inside a VM or container? (`none` = bare metal)
`lscpu | grep -i hypervisor`	shows the hypervisor vendor inside a VM
`grep -Ec 'vmx|svm' /proc/cpuinfo`	above 0 = CPU supports hardware virtualization (Intel vmx, AMD svm)
`sudo dmidecode -s system-product-name`	e.g. VirtualBox, VMware Virtual Platform

DOCKER BASICS

`docker run -it --rm ubuntu bash`	throwaway Ubuntu container shell
`docker ps -a`	running and stopped containers
`docker images`	downloaded images
`docker exec -it <name> bash`	shell into a running container

VM vocabulary: hypervisor (VMM) runs VMs · host OS = the real one · guest OS = inside the VM · virtual disk (.vdi, .vmdk, .qcow2) · **snapshot** = save point (take one before every risky lab step) · template = pre-built VM to clone. **Paravirtualization** (Xen) = guest OS is modified to cooperate with the hypervisor. **Emulator** = pretends to be different hardware (slower).

11. Security quick checks

DEFENSES AND NETWORK

`getenforce / sestatus`	SELinux mode: Enforcing, Permissive, Disabled (RHEL/Fedora)
`sudo aa-status`	AppArmor profiles (Ubuntu/Debian)
`sudo ufw status verbose`	firewall (Ubuntu)
`sudo firewall-cmd --list-all`	firewall (RHEL/Fedora)

WATCH AND PATCH

`who / w / last`	who's logged in, and login history
`sudo lastb`	failed logins
`sudo grep 'Failed password' /var/log/auth.log`	SSH brute force (use /var/log/secure on RHEL)
`sudo journalctl -u ssh --since today`	one service's log (unit may be `sshd`)

`sudo iptables -L -n -v` / `sudo nft list ruleset`	raw firewall rules
`ss -tulpn`	listening ports and which process owns each
`ip a` / `ip r`	interfaces and routes

`awk -F: '$3==0' /etc/passwd`	accounts with UID 0 (should only be root)
`sudo apt update && sudo apt upgrade`	patch (Debian/Ubuntu); `sudo dnf upgrade` on RHEL/Fedora

Attack	Level	Main defenses
Buffer overflow	kernel or process	patching, secure coding, ASLR and stack protection (on by default)
Privilege escalation	kernel or process	least privilege, audit SUID files and `sudo -l`, patch the kernel
Code injection / process hijacking	kernel or process	input validation, SELinux/AppArmor, monitoring
Denial of service	kernel	resource limits, firewall, monitoring
Zero-day	any	defense in depth: isolation (VMs/containers), IDS, logging, trusted sources

12. Common mix-ups

- **Block size:** disks send data in 512-**byte** sectors (4 KB on most modern disks). Filesystems usually use 4 KB blocks.
- **Command names:** `tune2fs` (not tunefs2), `mknod` (not makenod).
- **chmod letters:** `u` = owner (user), `o` = **others**. Sticky bit: `chmod +t dir` or `chmod 1777 dir`.
- **Finding SUID:** `-perm -4000` = SUID, `-perm -6000` = SUID *and* SGID, `-perm /6000` = either one.
- **Swap size:** "2x RAM" is an old rule of thumb, not something Linux does on its own. Many modern systems use a swap file or less swap.
- **Docker vs Kubernetes:** Docker is made by Docker, Inc. Kubernetes (started at Google, now run by the CNCF) *manages* containers rather than being one.
- **Xen** is the hypervisor; Citrix sold it as XenServer (later Citrix Hypervisor).
- **Architectures:** macOS ran on Intel x86 for years before Apple Silicon, and Windows also runs on ARM.
- **Paging vs swapping:** paging splits memory into fixed-size pages. Moving inactive pages to disk is **swapping** (paging out).
- **Containers vs VMs:** containers share one kernel, so a kernel exploit can escape them. VMs isolate more strongly, but configuration matters for both.